# Del-York Website Notes

A detailed explanation of how the website works, what the HTML tags do, and how the CSS styles control the design.

---

## 1. Website Overview

The website has two main pages and one main stylesheet.

- **index.html** is the landing page. It contains the hero section, about section, subsidiaries carousel, footer, custom cursor, background effects, and carousel JavaScript.
- **contact.html** is the contact page. It shares the same header, footer, background blobs, and cursor effect, but adds office information and a contact form.
- **style.css** controls the visual design, spacing, typography, animations, responsiveness, hover effects, carousel styling, and contact page design.

## 2. Basic HTML Structure

```
<!DOCTYPE html>
```

This tells the browser the page uses modern HTML5.

```
<html lang="en">
```

This wraps the entire page. The `lang="en"` attribute tells browsers and screen readers that the page language is English.

```
<head> ... </head>
```

The head contains information about the page that is mostly not visible on the screen.

```
<title>Del-York Group Landing Page</title>
```

This sets the browser tab title.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

This makes the page responsive on phones and tablets by matching the layout width to the device width.

```
<link rel="stylesheet" href="style.css">
```

This connects the HTML page to the CSS file.

```
<body> ... </body>
```

The body contains all visible page content: header, navigation, sections, cards, forms, footer, and scripts.

## 3. Shared Visual Elements

```
<div class="custom-cursor"></div>
<div class="cursor-follower"></div>
```

These two empty divs become the custom mouse cursor. CSS gives them shape and color, while JavaScript moves them based on the mouse position.

```
<div class="bg-blobs">
  <div class="blob blob-1"></div>
  <div class="blob blob-2"></div>
  <div class="blob blob-3"></div>
</div>
```

These divs create the glowing red background shapes. CSS makes them circular, blurred, transparent, and animated.

## 4. Header and Navigation

```
<header> ... </header>
```

The header is the top area of the website. It contains the logo and navigation menu.

```
<div class="logo">
  <a href="#hero"><img src="cropped-Del-York_logo.png" alt="Company Logo"></a>
</div>
```

The logo is placed inside a link, so users can click it. The image includes an `alt` attribute, which describes the image for accessibility.

```
<nav class="main-nav">
  <ul>
    <li><a href="#hero" class="active">Home</a></li>
  </ul>
</nav>
```

`nav` represents a navigation area. `ul` is an unordered list, `li` is a list item, and `a` is a clickable link. The `active` class marks the current page or section.

## 5. Landing Page Sections

### Hero Section

```
<section class="hero" id="hero">
```

The hero is the first major area of the landing page. The `id="hero"` allows links like `#hero` to scroll to it.

```
<div class="hero-backdrop" aria-hidden="true">
```

This holds decorative background images. `aria-hidden="true"` tells assistive technology to ignore it because it is not important content.

```
<span class="hero-kicker">Creative capital. Industrial scale.</span>
```

A small label above the main heading. `span` is useful for styling a small inline piece of content.

```
<h1>A living ecosystem for Africa's next leap.</h1>
```

The main heading of the page.

```
<p>Del-York Group turns ideas...</p>
```

A paragraph of supporting text.

```
<a href="https://delyorkgroup.com/" class="cta-button">Visit Official Website</a>
```

This is a link styled as a call-to-action button.

## About Section

```
<section class="about-section" id="about-text">
```

This section explains the brand. The `id` allows the navigation link to jump directly to it.

```
<div class="about-visual"> ... </div>
<div class="about-content"> ... </div>
```

The about section is split into a visual side and a text side.

```
<h2>Where African ambition becomes world-ready enterprise.</h2>
```

A second-level heading used for a major page section.

## Subsidiaries Section

```
<section class="subsidiaries" id="subsidiaries">
```

This section displays the carousel of Del-York subsidiaries.

```
<button class="carousel-btn prev-btn" id="prevBtn" aria-label="Previous slide">
```

This button moves the carousel backward. The `aria-label` explains the button to screen readers.

```
<div class="carousel-container" id="carouselContainer">
  <div class="carousel-track"> ... </div>
</div>
```

The container is the scrollable area. The track holds all carousel cards in a horizontal row.

```
<div class="carousel-item" data-label="Creative">
```

Each carousel card has a `data-label` value. CSS uses this value to display the small label at the top of each card.

## 6. Contact Page Structure

```
<body class="contact-page-body">
```

The contact page adds a class to the body so contact-specific styles can apply.

```
<section class="contact-hero">
```

This is the top intro section of the contact page.

```
<div class="contact-grid">
```

This creates a two-column layout: contact information on the left and the form on the right.

```
<a href="mailto:info@delyorkgroup.com">
```

This opens the user's email app.

```
<a href="tel:+2349047721762">
```

This allows phones and supported devices to call the number.

```
<form class="premium-form">
```

This creates the contact form. At the moment, the form has no backend or `action` attribute, so it does not actually send the message anywhere yet.

```
<input type="text" id="name" name="name" required placeholder=" ">
<label for="name">Your Full Name</label>
```

The input collects text. The label is connected to it through the matching `for` and `id` values. The blank placeholder helps the floating label CSS work.

## 7. CSS Variables

```
:root {
  --bg-black: #050505;
  --accent-red: #A40000;
  --text-main: #FFFFFF;
  --font-oswald: 'Oswald', sans-serif;
}
```

`:root` stores global CSS variables. These values can be reused throughout the stylesheet with `var(...)`.

```
color: var(--text-main);
```

This uses the value stored in `--text-main`. Variables make the design easier to update.

## 8. CSS Reset and Page Defaults

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

The universal selector `*` applies to every element. This removes browser default spacing and makes element sizing easier to control.

```
html {
  scroll-behavior: smooth;
}
```

This makes anchor links scroll smoothly instead of jumping suddenly.

```
body {
  background: var(--bg-black);
  color: var(--text-main);
  font-family: var(--font-inter);
  line-height: 1.6;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  overflow-x: hidden;
  cursor: none;
}
```

The body styles set the page background, default text color, font, readable line spacing, full-screen minimum height, vertical layout, hidden horizontal overflow, and custom cursor behavior.

## 9. Pseudo-elements

```
body::before
.hero::after
.subsidiaries::before
.carousel-item::before
```

Pseudo-elements create extra visual layers without adding more HTML. For example, `body::before` adds the subtle texture overlay, and `.carousel-item::before` displays the card label.

```
.carousel-item::before {
  content: attr(data-label);
}
```

This pulls text from the HTML attribute `data-label` and displays it through CSS.

## 10. Background Blobs

```
.bg-blobs {
  position: fixed;
  width: 100%;
  height: 100%;
  z-index: -1;
  pointer-events: none;
}
```

The blob container covers the whole screen and sits behind the main content.

```
.blob {
  position: absolute;
  filter: blur(100px);
  border-radius: 50%;
  opacity: 0.15;
  animation: float 20s infinite alternate;
}
```

The blobs are soft circular shapes. The blur makes them glow, opacity makes them subtle, and animation moves them slowly.

## 11. Header CSS

```
header {
  display: flex;
  position: sticky;
  top: 0;
  z-index: 1000;
  height: 90px;
}
```

The header uses flexbox to place the logo and nav side by side. Sticky positioning keeps it visible at the top while scrolling.

```
.main-nav ul {
  list-style: none;
  display: flex;
  gap: 3.5rem;
}
```

This removes bullet points and places navigation links in a row with spacing between them.

## 12. Hero CSS

```
.hero {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(360px, 0.9fr);
  gap: clamp(2.4rem, 5.6vw, 5.6rem);
}
```

The hero uses CSS Grid to create two columns: text on the left and visuals on the right. `clamp()` creates responsive spacing with a minimum, preferred value, and maximum.

```
.hero h1 {
  font-size: clamp(4rem, 8vw, 8.7rem);
  background: linear-gradient(...);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}
```

This makes the main heading responsive and gives it a gradient text effect.

## 13. Buttons and Links

```
.cta-button {
  display: inline-block;
  padding: 1.2rem 3.5rem;
  background: var(--text-main);
  color: var(--bg-darker);
  border-radius: 100px;
  transition: all 0.4s;
}
```

The call-to-action link is styled like a pill-shaped button. The transition makes hover changes smooth.

```
.cta-button:hover {
  transform: translateY(-5px) scale(1.05);
}
```

On hover, the button moves upward and grows slightly.

## 14. About Section CSS

```
.about-section {
  display: grid;
  grid-template-columns: minmax(320px, 0.85fr) minmax(0, 1fr);
  scroll-margin-top: 100px;
}
```

The about section uses a two-column grid. `scroll-margin-top` prevents the sticky header from covering the section when users click an anchor link.

```
.about-points {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
```

This creates three equal columns for the Create, Build, and Connect blocks.

## 15. Carousel CSS

```
.carousel-container {
  overflow-x: auto;
  scroll-behavior: smooth;
  scroll-snap-type: x mandatory;
}
```

The carousel scrolls horizontally and snaps cards neatly into position.

```
.carousel-track {
  display: flex;
  gap: 2.4rem;
  width: max-content;
}
```

The carousel track places all cards in a horizontal row.

```
.carousel-item {
  width: 440px;
  height: 580px;
  flex-shrink: 0;
  scroll-snap-align: center;
}
```

Each card has a fixed size, does not shrink, and snaps to the center when scrolling.

## 16. Contact Page CSS

```
.contact-grid {
  display: grid;
  grid-template-columns: 1fr 1.5fr;
}
```

The contact page uses two columns, with the form column wider than the information column.

```
.glassmorphism {
  background: var(--glass-bg);
  backdrop-filter: blur(20px);
  border: 1px solid var(--glass-border);
}
```

This creates the glass-like form panel.

```
.form-group input:focus ~ label,
.form-group input:not(:placeholder-shown) ~ label {
  top: -1.2rem;
  font-size: 0.75rem;
}
```

This creates the floating label effect. When the input is focused or filled, the label moves upward and becomes smaller.

## 17. Responsive Design

```
@media (max-width: 768px) {
  .hero {
    grid-template-columns: 1fr;
  }
}
```

Media queries change the layout on smaller screens. This example changes the hero from two columns to one column on phones.

```
@media (hover: none), (pointer: coarse) {
  body {
    cursor: auto;
  }

  .custom-cursor,
  .cursor-follower,
  .ripple {
    display: none;
  }
}
```

This disables custom cursor effects on touch devices, where they would not make sense.

## 18. JavaScript Summary

The JavaScript in `index.html` does three main things:

- Creates carousel pagination dots.
- Moves the carousel automatically every three seconds.
- Controls the custom cursor, moving background blobs, and click ripple effect.

```
const container = document.getElementById('carouselContainer');
const track = document.querySelector('.carousel-track');
const prevBtn = document.getElementById('prevBtn');
const nextBtn = document.getElementById('nextBtn');
```

These lines select HTML elements so JavaScript can control them.

```
container.scrollBy({ left: getScrollAmount(), behavior: 'smooth' });
```

This scrolls the carousel smoothly by one card width.

```
setInterval(() => { ... }, 3000);
```

This runs the autoplay logic every three seconds.

## 19. Important Issue Noticed

> Some text in `index.html` has encoding problems, such as `Hollywoodâ€‘grade`, `Africaâ€™s`, and `stateâ€‘ofâ€‘theâ€‘art`. These should probably be `Hollywood-grade`, `Africa's`, and `state-of-the-art`.
>
> Adding `<meta charset="UTF-8">` inside the `<head>` of `index.html` can help prevent this type of issue.

## 20. Simple Summary

The website is a modern two-page landing site. HTML creates the structure: header, hero, sections, cards, contact form, and footer. CSS creates the visual design: black and red theme, typography, spacing, grids, animations, responsive layouts, glass effects, and hover states. JavaScript adds

interaction: carousel movement, pagination dots, custom cursor, background movement, and click ripple effects.